

데이터셋 준비 및 코드 작동 방법

이 문서는 추론과 학습에 필요한 데이터를 어떻게 준비하는지 설명합니다. 추론은 대회 테스트 폴더를 그대로 쓰므로 준비할 것이 없습니다. 학습은 원본 train 데이터를 한 번 변환해야 합니다. 명령은 모두 `etri_e2e_final/` 폴더에서 실행하고, Docker 이미지 `etri-e2e-ext:final` 안에서 동작합니다(README 3절).

1. 전체 흐름

```
[추론] 대회 테스트 폴더 (1,125 clip) → scripts/infer.sh → OUT_DIR/package/submission.zip

[학습] 대회 train 원본 (<scenario>.tar × 376)
      ↳ scripts/prepare_train.sh → PREP/etri_768 (768×432 영상), PREP/meta_train (parquet)
                                     |
동봉 파생 캐시 (data/) —————→ |
동봉 시작 checkpoint (checkpoints/) —→ scripts/train.sh 1 | 2 | 3 → OUT_DIR/stage*/
```

2. 추론용 테스트 데이터

대회에서 배포한 테스트 폴더를 그대로 사용합니다. 폴더 안에 clip 1,125개가 있어야 하고, 각 clip은 다음 구성입니다.

```
<clip_hash>/
├ camera_front/frame_0.jpg ... frame_-30.jpg      (6개 카메라 폴더 모두)
├ ego_pose.parquet                                frame -30 ~ 0 과 +50 (목표점)
└ calibration.parquet
```

추론 코드가 전처리를 모두 수행하므로 별도로 준비할 것이 없습니다.

- 카메라별 왜곡 보정, crop(1920×1080), 축소(768×432, 과거 영상은 384×216)
- 과거 pose로 현재 ego 상태 5값 계산
- +50 frame 목표점 읽기

```
bash scripts/infer.sh /path/to/test OUT_DIR      # RTX 4090 약 7분
```

3. 학습 데이터

3.1 원본 → 학습 입력 (scripts/prepare_train.sh)

학습 코드는 원본 영상을 직접 읽지 않고, 미리 변환한 768×432 영상과 추출한 parquet을 읽습니다. 매 step마다 원본 1920×1536 영상 6장을 왜곡 보정하면 CPU가 병목이 되므로, 한 번만 변환해 두는 구조입니다.

```
bash scripts/prepare_train.sh /path/to/train PREP      # 결과 약 70 GB, 1회
JOBS=24 bash scripts/prepare_train.sh /path/to/train PREP  # 병렬 작업 수 지정 (기본 8)
```

- **입력:** 대회 제공 train 376 scenario입니다. 배포 형식인 `<scenario>.tar` 파일 376개가 든 폴더를 지정합니다(약 199 GB). 압축을 푼 `<scenario>/` 폴더도 받습니다. 각 scenario에는 6개 카메라 영상 1,800장과 `annotation/`, `calibration/`, `meta/`의 parquet이 있습니다.
- **출력:**

출력 (PREP/)	내용	학습 시 컨테이너 경로
etri_768/<scenario>/<camera>/<frame>.jpg	왜곡 보정 → 1920×1080 crop → 0.4배 → JPEG 품질 95 (768×432), 676,800장	/tmp/pm97/cache/etri_768
etri_768/cache_meta.json	scenario·카메라별 crop 위치와 변환된 카메라 행렬	(같은 폴더)
meta_train/<scenario>/{annotation,calibration,meta}/	원본 parquet 그대로, 3,008개	/tmp/pm97/data/etri/meta_train

- **crop 기준:** 가로는 중앙을 기준으로 자릅니다. 세로는 전방과 후방 광각 카메라는 하단을, 측방 4개 카메라는 상단을 기준으로 자릅니다.
- **코드:** 영상 변환은 학습 캐시를 만들 때 쓴 `src/data_prep/etri_build_cache.py`의 함수를 그대로 사용합니다 (`tools/build_train_cache.py`). 중간에 멈춰도 다시 실행하면 이미 만든 파일은 건너뛵니다.
- **확인 결과:** 원본 376 scenario 전체로 다시 만든 결과를 학습에 사용한 캐시와 비교했습니다. 영상 676,800장, parquet 3,008개, crop과 카메라 행렬이 모두 바이트 단위로 같습니다.
- `scripts/train.sh`는 두 번째 인자인 PREP의 두 폴더를 위 컨테이너 경로에 읽기 전용으로 마운트합니다.

3.2 파생 캐시 (동봉)

train 원본에서 미리 계산해 둔 캐시입니다. 학습 코드가 이 파일을 읽으며, 동봉 파일을 그대로 사용합니다.

파일 (data/)	크기	내용	생성 코드 (src/data_prep/)
ego_cache.npz	45 MB	376 scenario × 300 프레임의 ego 과거·미래 궤적, 목표점, 속도·가속도	etri_ego_cache.py
ego_cache_5s.npz	8 MB	5초 10점 정답 궤적 (3단계 후보 배정용)	etri_label_5s.py
split.json	0.2 MB	시간 연속 session 단위 train/tune/val 분할	build_grouped_split_v2.py
supervision/	54 MB	scenario별 occupancy-lane-ego 상태/과거 궤적 감속 정답과 카메라 보정	build_scene_supervision_v2.py, derive_motiondrive_v2_geometry.py
status_full/	2 MB	전체 376 scenario의 행별 ego 상태 5값 (과거 pose 5개로 계산)	build_full_cache.py (src/h4_status.py 사용)
status_dev/	2 MB	같은 값의 train/tune 분할 캐시 (학습 코드가 참조)	src/h4_status.py
status_nominal/	2 MB	이전 단계 방식(프레임 차분)의 상태 입력 캐시 (학습 코드가 참조)	build_nominal_inputs.py
metadata/	4 MB	학습 코드가 읽는 이전 단계의 실행 설정과 검증 기록	—
fixtures_train8/	26 MB	검증용 학습 clip 8개 (테스트 폴더와 같은 형식)	—

- 생성 순서는 원본 → ego 캐시 → 5초 정답 → 분할 → scenario 감속 → geometry 파생 → 상태 캐시입니다.
- 생성 코드는 사용한 그대로 동봉합니다. 다만 일부 코드는 당시의 중간 산출물(카메라 보정 pkl, 과거 검증 분할 등)을 입력으로 읽으므로, 동봉 캐시를 그대로 사용하는 것을 권장합니다.
- 각 폴더의 manifest에 파일별 sha256이 기록되어 있고, 학습 코드는 시작할 때 이 해시를 확인합니다.

4. Checkpoint (checkpoints/)

파일	용도
backbone_nuimages_cascade_r50.pth	공개 백본 (외부 데이터, README 9절)
stage1_initializer.pth	1단계 시작점 (공개 백본 + 무작위 초기화, 대회 데이터 학습 전)
stage1_h4_progress_full.pth (+ .experiment.json)	1단계 결과이자 2단계 시작점
stage2_l_full6.pth (+ .experiment.json)	2단계 결과이자 3단계 시작점
ext_full_v7.pth	제출 모델 (3단계 결과, 리더보드 0.117067)

2단계 코드는 시작 checkpoint의 sha256을 확인하므로 동봉한 1단계 결과에서 시작합니다. 3단계도 동봉한 2단계 결과에서 시작합니다.

5. 코드 작동 순서

명령	컨테이너 안에서 실행되는 코드
validate.sh	tools/validate_release.py: 모델을 불러와 동봉 clip 28개를 추론하고, FLOPs와 forward 시간을 측정합니다
infer.sh	src/build_ext_submission.py로 1,125 clip을 추론·병합하고 FLOPs를 기록합니다. 이어서 src/package_submission.py가 제출 zip을 만듭니다
prepare_train.sh	tools/build_train_cache.py: 3.1절의 변환을 수행합니다
train.sh 1	src/train_full.py: 1단계 (24,931 update)
train.sh 2	src/repro_lfull6.py: smoke(5 update) → verify(재로드·BN·입력 경로 점검) → main(38,070 update)
train.sh 3	src/train_ext.py: 3단계 (6,344 update)

모든 명령은 먼저 tools/assemble.py로 컨테이너 안에 원래 저장소 경로를 구성한 뒤 실행됩니다(README 8절).